Supplementary Information

Time-domain noise as an evaluation axis for raw-signal nanopore mapping

Khushal Singh[1,*] and Ritu Garg[1]

[1]Department of Computer Engineering, National Institute of Technology Kurukshetra, Kurukshetra, Haryana 136119, India

[*]To whom correspondence should be addressed: khushal6002@gmail.com

This supplement has three parts. Sections S1–S3 report three observations made during this work that rest in part on a measurement for which no run output was captured. Those values are marked with a dagger (†). None of them is used in any claim in the main text, and `scripts/verify_dwell_journal_numbers.py` lists each as outstanding, together with the run that would settle it. Sections S4 and S5 give committed results in more detail than the main text does, and every value in them is recomputed by the same script from files in the repository. Section S6 documents the use of AI tools in this work. Section and table numbers without an S refer to the main text.

S1 Live accept rates against Icarust

With the same client, reference and run length, the fixed-width arm accepted 43.5%† of reads and the detected arm 9.6%† against stock Icarust, and 0.5%† and 9.6%† against the patched simulator at CV 0.35. Taken at face value, stock Icarust reports the fixed-width arm as 4.5-fold† better, and the patched simulator reverses the order, which is the reversal of Section 3.1 observed on the live interface. These are accept rates on the live loop and are not comparable with the offline true-positive rates (TPR) of Table 2. Re-running stock and patched Icarust against the client would settle them.

S2 Calibration target for event detection

The calibrated detector, threshold 3.0, emits 1562 events per on-target read at CV 0.44, against an ideal of approximately 2000, one event per base, which is 21.9% under-segmentation; seed 42 gives 14.2% TPR in the captured log. In a development run that was not captured, threshold 2.0 gave 2017† events per read, close to the ideal, and 2.0%† TPR, 7.1-fold† lower. An argument offered during development attributed the difference to frame displacement: a spurious boundary shifts every later key in the read. That argument does not distinguish the two error types, because a missed boundary removes an event and shifts every later event index by one exactly as a spurious boundary adds one. A sufficient explanation is simpler. The event count measures net error, insertions minus deletions, and is blind to errors that cancel, so a detector that emits the ideal number of events may still place many boundaries wrongly. The recommendation that follows does not depend on the uncaptured values: calibrate a detector against agreement or indel fraction (Section 2.6), not against its event count.

S3 Calibration transfer between generators

A detector threshold calibrated against signal from a generator written for this project, applied unchanged to Squigulator signal, gave 0.6%† TPR at CV 0, where the Squigulator-calibrated detector gives 18.4% [17.0–19.8] (Table 2), a 30.7-fold† difference. The two generators differ in their amplitude-noise models,

and the detector threshold is a t-statistic whose operating point depends on that model. Whatever the magnitude, a result obtained with event detection is interpretable only together with a statement of how the detector was calibrated and on what signal.

S4 RawHash2 at matched operating points

RawHash2's results in Tables 3 and 5 are at its default operating point, which maps some control reads. Tables S1 and S2 place it at two operating points set by a false-positive budget, from the output of the dwell × window grid (Section 2.5) and without new runs. `scripts/rawhash_zero_fp.py` reads that output from `results/rawhash-grid-raw/raw.tar.gz`. It first scores every cell with no threshold, by the rule of Section 2.5, and requires the result to reproduce `results/rawhash-grid-seeds.csv` exactly, which all 54 cells do. It then thresholds the mapping quality that RawHash2 reports for each mapping, the twelfth column of its PAF output. At zero false positives (Zero FP) the threshold is one above the highest mapping quality given to any control read. With a 1% budget (1% FP) it is the lowest threshold that leaves at most 5 of the 500 control reads mapped, so it is set by the sixth-highest-scoring control read. An on-target read counts if it is mapped correctly with a mapping quality at or above the threshold. Each seed's thresholds and rates are in `results/rawhash-grid-zero-fp.csv`, which the verification script recomputes from the raw output on every run.

Table S1. RawHash2's correct-mapping rate (%) at matched operating points, by dwell CV and minimizer window w: the percentage of on-target reads mapped correctly at or above the lowest mapping-quality threshold that admits no control read (Zero FP) or at most 1% of them (1% FP). Mean [range] over three seeds, 500 on-target and 500 control reads per cell.

Operating point	Dwell CV	$w = 0$	$w = 5$	$w = 10$
Zero FP	0.00	86.3 [85.2–87.0]	79.7 [76.8–84.8]	70.6 [67.4–73.4]
Zero FP	0.10	88.7 [86.4–91.2]	82.8 [77.8–85.6]	70.1 [63.2–75.2]
Zero FP	0.20	89.5 [87.4–92.8]	80.3 [76.6–82.6]	67.7 [64.2–73.6]
Zero FP	0.30	83.1 [81.6–85.8]	77.8 [75.2–82.6]	65.3 [63.0–68.4]
Zero FP	0.44	80.5 [76.2–84.6]	78.1 [75.0–82.8]	62.5 [60.2–66.8]
Zero FP	0.60	82.1 [79.4–86.4]	71.7 [70.4–73.8]	52.5 [41.8–62.8]
1% FP	0.00	90.6 [89.8–91.6]	86.8 [85.6–88.2]	77.2 [75.8–78.8]
1% FP	0.10	91.4 [90.6–92.8]	86.4 [84.6–87.6]	78.5 [78.2–78.6]
1% FP	0.20	91.1 [90.0–93.0]	86.0 [85.4–86.8]	76.7 [73.6–78.4]
1% FP	0.30	89.5 [87.0–92.6]	84.7 [83.8–86.0]	74.2 [70.8–76.6]
1% FP	0.44	88.4 [87.6–89.0]	82.5 [81.2–84.2]	71.9 [70.2–73.6]
1% FP	0.60	87.5 [86.2–89.0]	77.9 [76.6–79.4]	63.0 [61.2–65.8]

Three features of these tables bear on the main text. First, a zero-false-positive threshold rests on the single highest-scoring control read, and the rates at it are correspondingly noisy: at $w = 0$ the Zero FP rate is not monotone in CV, and at CV 0.44 one seed's $w = 5$ index maps 6.6 points more of its reads correctly than its $w = 0$ index, which no seed does with the 1% budget or at the default (Table S3). Second, at $w = 10$ sketching costs more at both matched operating points than at the default, at every CV (Tables 5 and S2). Third, the dependence of that cost on dispersion (Section 3.3) holds with the 1% budget at both windows, in every seed and with disjoint ranges between CV 0 and CV 0.60; at zero false positives it holds in every seed at $w = 10$ only.

S5 Per-seed sketching cost at the default operating point

Table S3 gives each seed's cost of sketching at RawHash2's default operating point, the values that Table 5 summarises and Figure 4 draws: the correct-mapping rate at $w = 0$ minus that at the stated window, from

Table S2. Accuracy cost of minimizer sketching in RawHash2 at the operating points of Table S1: the rate at $w = 0$ minus that at the stated window, in percentage points, taken within each seed; mean [range] over three seeds. A negative value means that the sketched index mapped more reads correctly in that seed.

Operating point	Dwell CV	$w = 5$	$w = 10$
Zero FP	0.00	6.7 [0.4–10.0]	15.7 [13.4–19.6]
Zero FP	0.10	5.9 [2.8–8.6]	18.6 [13.2–23.2]
Zero FP	0.20	9.1 [4.8–11.6]	21.8 [13.8–27.6]
Zero FP	0.30	5.3 [3.2–6.4]	17.9 [13.6–21.4]
Zero FP	0.44	2.5 [−6.6–8.2]	18.0 [9.4–24.0]
Zero FP	0.60	10.4 [9.0–12.6]	29.5 [16.6–38.6]
1% FP	0.00	3.8 [3.4–4.2]	13.4 [12.8–14.6]
1% FP	0.10	5.0 [3.2–6.0]	12.9 [12.0–14.6]
1% FP	0.20	5.1 [3.6–7.2]	14.4 [11.8–19.4]
1% FP	0.30	4.7 [2.6–8.8]	15.3 [10.4–21.8]
1% FP	0.44	5.9 [3.4–7.4]	16.5 [15.0–17.4]
1% FP	0.60	9.6 [8.6–10.6]	24.5 [21.4–27.0]

`results/rawhash-grid-seeds.csv`. Every window of one seed maps the same reads, so each entry is a paired difference. In each seed the cost at CV 0.60 exceeds the cost at CV 0 at both windows, which is the claim of Section 3.3. Only seed 44's cost rises at every step of the sweep, at both windows, which is why that section does not claim a rise at every step.

Table S3. Accuracy cost of minimizer sketching in RawHash2 at its default operating point, in each seed: correct-mapping rate at $w = 0$ minus that at the stated window, in percentage points. Table 5 gives the mean and range of each set of three.

	$w = 5$			$w = 10$		
Dwell CV	Seed 42	Seed 43	Seed 44	Seed 42	Seed 43	Seed 44
0.00	5.0	5.0	4.0	12.8	11.8	10.4
0.10	6.4	5.2	4.6	13.0	11.4	10.6
0.20	5.2	3.6	5.2	12.4	10.4	10.8
0.30	6.2	4.0	5.8	15.0	10.2	11.6
0.44	7.2	5.4	6.6	15.2	16.4	14.6
0.60	10.4	7.2	8.6	21.0	18.2	19.2

S6 Use of AI tools

The authors used Claude (Anthropic) through Claude Code, a tool that reads and edits files and runs commands in the authors' environment at their direction; the models were Claude Opus 5 and Claude Opus 5.5. The repository's history records the extent of the use: every commit in it carries a Claude co-author line, including those that add the pipeline's source code, the analysis scripts and the dissertation drafts. Its tasks for this article were the following.

Code and analysis. It helped write the pipeline's source code and the scripts that map and score reads, sweep dwell dispersion and draw the figures. While this article was prepared it ran the three-seed RawHash2 grid of Section 2.5 (`scripts/rerun_rawhash_grid.sh`) on simulated reads generated earlier in the work, and it wrote and ran `scripts/rawhash_zero_fp.py`, which computes Section S4 from that grid's output. No reads were simulated while this article was prepared.

Verification. It wrote `scripts/verify_dwell_journal_numbers.py`, which recomputes every number in the article and in this supplement from the committed result files, and tested the script by planting errors in copies of both documents, each of which the script had to reject. Checking the dissertation chapter's numbers against the raw data in this way led to several corrections, including the replication over three seeds of a result first reported from one.

References. It identified candidate references and checked each against Crossref, the publisher's page or arXiv.

Text and figures. It drafted this article from Chapter 6 of the first author's dissertation, which it had also helped draft, and typeset the LaTeX source. Figure 1 is a schematic drawn by a script it wrote.

The authors directed each task, reviewed the code, the data and the text, and take full responsibility for the content. No AI tool is an author.